



GURU NANAK DEV ENGINEERING COLLEGE

DEPARTMENT OF
INFORMATION TECHNOLOGY

DATA STRUCTURES

Notes

UNIT 2 (ARRAY)

0	1	2	3	4
10	25	7	18	32



Subject Teachers

- Prof. Parminder Kaur Wadhwa
- Prof. Ranjodh Kaur
- Prof. Gitanjali

D2IT

Second Year Students



GURU NANAK DEV
ENGINEERING
COLLEGE

DEPARTMENT OF
INFORMATION
TECHNOLOGY

*Better
Logic
Builds a
Brighter
Future*



DATA
STRUCTURES

UNIT 2 (ARRAY)

0	1	2	3	4
10	25	7	18	32

D2IT
Second Year
Students

Subject Teachers

- Prof. Parminder Kaur Wadhwa
- Prof. Ranjodh Kaur
- Prof. Gitanjali



Unit-2 Array

Linear and multidimensional arrays and their representation, insertion, deletion and searching in arrays, Comparison of performance of Binary Search algorithm and Linear Search algorithm, Efficient storage methods for sparse matrices, Limitations of Array as a data structure, Applications of array in real life:- storing data, image processing, financial and statistical analysis, data analysis in machine learning, social media analytics; Sort the elements in an array using quicksort, mergesort and heapsort.

Arrays

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations.

Example:

```
A = [10, 20, 30, 40, 50]
```

Here all elements are integers and stored one after another in memory.

```
Array: 10 20 30 40 50
```

```
Index: 0 1 2 3 4
```

The position of an element is called its index.

- $A[0] = 10$
- $A[1] = 20$
- $A[2] = 30$

Types of Arrays

1. Linear (One-Dimensional) Array

A linear array stores elements in a single row or single line.

Example:

```
A = [5, 10, 15, 20, 25]
```

Representation:

```
Array: 5 10 15 20 25
```

```
Index: 0 1 2 3 4
```

C/C++ declaration:

C++

```
int A[5] = {5,10,15,20,25};
```

Memory Representation

Assume each integer takes 4 bytes.

Base address = 1000

Index	Value	Memory Address
0	5	1000
1	10	1004
2	15	1008
3	20	1012
4	25	1016

Notice the addresses are continuous.

Address Formula

For a linear array:

$\text{Address}(A[i]) = \text{Base Address} + i \times (\text{size of each element})$

Example:

Base = 1000

Size of int = 4 bytes

Address of A[3]: $1000 + 3 \times 4 = 1012$

2. Multidimensional Array

A multidimensional array stores data in rows and columns.

The most common is the 2D array.

Example:

```
A = [ [1,2,3],  
      [4,5,6] ]
```

Representation:

```
1 2 3  
4 5 6
```

It has 2 rows and 3 columns.

C/C++ declaration:

C++

```
int A[2][3] = {{1,2,3},  
               {4,5,6}};
```

Accessing elements:

- $A[0][0] = 1$
- $A[0][2] = 3$
- $A[1][1] = 5$

Representation of 2D Array in Memory

Although it looks like a table, memory is actually one continuous line.

Row-major order (used in C/C++)

Elements are stored row by row.

Memory layout:

1 2 3 4 5 6

Addresses (assuming base = 1000 and int = 4 bytes)

Element	Address
A[0][0]	1000
A[0][1]	1004
A[0][2]	1008
A[1][0]	1012
A[1][1]	1016
A[1][2]	1020

Row-major Address Formula

For an array with m columns:

$\text{Address}(A[i][j]) = \text{Base} + ((i \times m) + j) \times \text{size}$

Example:

2 × 3 array

Base = 1000

Size = 4

Find address of A[1][2]:

$1000 + ((1 \times 3) + 2) \times 4$

$$1000 + (3+2) \times 4$$

$$1000 + 20 = 1020$$

Insertion in a Linear Array

Suppose the array has empty space.

Original array:

10 20 30 40 _

Insert 25 at index 2.

Step 1: Shift elements right

- Move 40
- Move 30

10 20 _ 30 40

Step 2: Insert 25

10 20 25 30 40

Algorithm

for i = n-1 down to position

$A[i+1] = A[i]$

$A[\text{position}] = \text{value}$

Time Complexity

Worst case: $O(n)$

(Because many elements may need shifting.)

Deletion from a Linear Array

Delete 20 from index 1.

Original:

```
10 20 30 40 50
```

Step 1: Shift elements left

Move:

- 30
- 40
- 50

```
10 30 40 50 _
```

Final array:

```
[10, 30, 40, 50]
```

Algorithm

```
for i = position to n-2
```

```
    A[i] = A[i+1]
```

```
n = n - 1
```

Time Complexity

Worst case: $O(n)$

Searching in Arrays

Searching means finding whether an element exists in the array.

There are two common methods.

1. Linear Search

Check each element one by one.

Array:

```
[10,20,30,40,50]
```

Search 40.

Steps:

- Compare with 10
- Compare with 20
- Compare with 30
- Compare with 40 Found

Algorithm

```
for i = 0 to n-1
  if A[i] == key
    return i

return -1
```

Time Complexity

- Best case: $O(1)$ (first element)
- Worst case: $O(n)$ (last element or not found)

2. Binary Search

Works only on a sorted array.

Array:

```
[10,20,30,40,50,60,70]
```


Search 50.

Steps:

1. Middle = 40
2. $50 > 40$ go right
3. Middle = 60
4. $50 < 60$ go left
5. 50 found

Time Complexity

- Best case: $O(1)$
- Worst case: $O(\log n)$

Much faster than linear search for large arrays.

Comparison of Operations

Operation	Time Complexity
Access by index	$O(1)$
Insertion	$O(n)$
Deletion	$O(n)$
Linear Search	$O(n)$
Binary Search	$O(\log n)$

Easy Memory Trick

Think of students sitting in a row.

- Insertion: Everyone moves one seat right.
- Deletion: Everyone moves one seat left.
- Linear Search: Ask each student one by one.
- Binary Search: Ask the middle student first, then half the class.

1. Sparse Matrices and Efficient Storage Methods

What is a sparse matrix?

A **matrix** is a collection of numbers arranged in rows and columns.

For example:

```
A =  
[ 0 0 0 5]  
[ 0 0 8 0]  
[ 0 0 0 0]  
[ 3 0 0 0]
```

This is a **4 × 4 matrix**, so it has 16 elements.

But notice that only **3 elements are non-zero**: 5, 8, 3 .

A matrix in which **most elements are zero** is called a **sparse matrix**.

Why is normal storage inefficient?

If we store the above matrix in a normal 2D array, we store all 16 values:

```
0 0 0 5 0 0 8 0 0 0 0 0 3 0 0 0
```

Most of the memory is being wasted on zeros.

Instead, we can store **only the non-zero values and their positions**.

Method 1: Triplet Representation

In this method, we store three things:

1. Row number

2. Column number
3. Value

For our matrix:

```
[ 0 0 0 5 ]  
[ 0 0 8 0 ]  
[ 0 0 0 0 ]  
[ 3 0 0 0 ]
```

We can store:

Row	Column	Value
0	3	5
1	2	8
3	0	3

Sometimes the first row also stores matrix information:

```
Rows  Columns  Non-zero elements  
4     4         3
```

Advantage

Instead of storing 16 numbers, we store information about only 3 non-zero numbers.

Example in real life

Suppose a university has **10,000 students and 10,000 courses**, but each student takes only about 5 courses.

A huge student-course matrix would contain millions of zeros.

A sparse representation stores only:

```
Student  Course  Enrollment  
101      CS101    1  
101      MA101    1
```

```
102    CS101    1
...
```

This saves a lot of memory.

Why use sparse storage?

The main benefits are:

- Less memory usage
 - Faster operations when most values are zero
 - Useful for very large datasets
 - Common in machine learning, graphs, scientific calculations, and search engines
-

2. Limitations of Array as a Data Structure

An **array** stores multiple elements of the same type in a sequence.

Example:

```
marks = [70, 85, 90, 65, 75]
```

Arrays are very useful, but they have some limitations.

1. Fixed Size

In many traditional array implementations, the size must be decided when the array is created.

Example:

```
int marks[5];
```

This array can hold only 5 elements.

If we later need 10 elements, we cannot simply add them to the same fixed-size array.

We may need to create a larger array and copy the old elements.

2. Insertion Can Be Slow

Suppose:

```
[10, 20, 30, 40]
```

We want to insert 15 between 10 and 20 .

We have to move elements:

```
[10, 20, 30, 40]
```

```
[10, 20, 20, 30, 40]
```

Then insert 15 :

```
[10, 15, 20, 30, 40]
```

So insertion in the middle can require moving many elements.

3. Deletion Can Be Slow

Suppose:

```
[10, 20, 30, 40, 50]
```

We remove 30 .

The remaining elements need to be shifted:

```
[10, 20, 40, 50]
```

Again, many elements may need to move.

4. Wasted Memory

If we create an array for 1,000 elements but actually use only 100, some memory may remain unused.

```
Array capacity = 1000  
Elements used = 100
```

This can waste memory.

5. Same Type of Data

Traditional arrays normally store elements of the same data type.

For example:

```
int numbers[5];
```

is designed to store integers.

It is not naturally designed for a mixture such as:

```
10, "John", 25.5, True
```

6. Searching May Be Slow

If an array is unsorted:

```
[45, 12, 89, 23, 67]
```

To find `67`, we may have to check:

```
45  12  89  23  67
```

This is called **linear search**, and in the worst case we may check every element.

3. Applications of Arrays in Real Life

Arrays are extremely important because they allow us to store and process large amounts of related data.

A. Storing Data

One of the simplest applications is storing data.

For example, the marks of five students:

```
marks = [78, 85, 91, 67, 88]
```

We can easily calculate:

- Total marks
- Average marks
- Highest marks
- Lowest marks

For example:

```
Total = 78 + 85 + 91 + 67 + 88  
       = 409
```

Arrays can also store:

[10, 20, 30, 40, 50]	prices
[25, 28, 31, 29, 27]	temperatures
[100, 200, 150, 300]	sales

4. Image Processing

An image can be represented using an **array of pixels**.

Black-and-white image

A simple black-and-white image can be represented as:

```
[0 0 1 1]  
[0 1 1 0]  
[1 1 0 0]  
[0 0 0 1]
```

For example:

```
0  black  
1  white
```

Each number represents one pixel.

Color images

Color images usually have three values for each pixel:

```
R  Red  
G  Green  
B  Blue
```

For example:

```
Pixel = [255, 0, 0]
```

means a red pixel.

So an image can be represented as a **3D array**:

```
Image  rows × columns × RGB channels
```

Applications

Arrays are used in:

- Image resizing
- Image rotation
- Brightness adjustment

- Face detection
 - Edge detection
 - Image filtering
 - Computer vision
-

5. Financial and Statistical Analysis

Arrays are heavily used for financial data.

Suppose a company's sales for five months are:

```
sales = [50000, 55000, 48000, 62000, 70000]
```

We can calculate:

Total sales

```
50000 + 55000 + 48000 + 62000 + 70000  
= 285000
```

Average sales

```
285000 / 5 = 57000
```

We can also find:

- Highest sales
- Lowest sales
- Growth rate
- Average profit
- Monthly expenses

Similarly, in statistics we can store:

```
heights = [165, 170, 172, 160, 175, 168]
```

and calculate the:

- Mean
 - Median
 - Maximum
 - Minimum
 - Standard deviation
-

6. Data Analysis in Machine Learning

Arrays are extremely important in **machine learning**.

Suppose we have information about students:

Age	Study Hours	Attendance
18	3	80
19	5	90
18	2	70
20	6	95

This can be represented as an array/matrix:

```
[  
  [18, 3, 80],  
  [19, 5, 90],  
  [18, 2, 70],  
  [20, 6, 95]  
]
```

Each **row** represents one student.

Each **column** represents one feature:

```
Column 1  Age  
Column 2  Study Hours  
Column 3  Attendance
```

Machine-learning algorithms process these numerical arrays to find patterns and make predictions.

Example

A machine-learning system could use:

```
Study hours + Attendance
```

to predict:

```
Exam score
```

Arrays are therefore fundamental to datasets, mathematical calculations, and model training.

7. Social Media Analytics

Social media platforms generate huge amounts of data.

Arrays can store information such as:

```
likes = [120, 450, 230, 890, 340]
```

or:

```
comments = [20, 35, 18, 90, 42]
```

Suppose five posts have:

```
likes = [100, 200, 500, 300, 150]
```

We can find the most popular post:

```
500  highest number of likes
```

Arrays can help analyze:

- Likes
- Comments
- Shares
- Followers
- Views
- Post engagement
- Hashtag frequency
- User activity

Example

A company might compare engagement:

```
Monday    1000 likes
Tuesday   1500 likes
Wednesday 2300 likes
Thursday  1800 likes
Friday    3000 likes
```

The company can see that **Friday had the highest engagement.**

8. Sorting an Array

Sorting means arranging elements in a particular order.

For example:

```
Unsorted:
[40, 10, 30, 20, 50]
```

Ascending order:

```
[10, 20, 30, 40, 50]
```

Three important sorting algorithms are:

1. Quicksort
2. Mergesort

9. Quicksort

Basic idea

Quicksort selects one element as a **pivot**.

It then divides the other elements into two groups:

- Elements smaller than the pivot
- Elements greater than the pivot

Then it recursively sorts those groups.

Example

Consider:

```
[40, 20, 60, 10, 30]
```

Quick Sort — Last Element as Pivot

```
pivot = 30
```

Initial positions

```
i = -1
```

```
j = 0
```

j **checks** every element **i** **moves** only when a smaller element is found pivot stays **at the last** until the end.

```
j
```

```
[40, 20, 60, 10, 30]
```

```
i      pivot
-1
```

Partitioning

The rule is:

- If $A[j] < \text{pivot}$ increase i and swap $A[i]$ with $A[j]$.
- If $A[j] \geq \text{pivot}$ only increase j .
- Pivot stays at the last position.

Step 1

```
j = 0
```

```
A[j] = 40
```

Is $40 < 30$?

No.

So only j moves.

```
i = -1
```

```
j = 1
```

Array:

```
[40, 20, 60, 10, 30]
```

Step 2

```
j = 1
```

```
A[j] = 20
```

Is $20 < 30$?

Yes.

So:

```
i = i + 1  
i = 0
```

Swap `A[i]` and `A[j]` :

```
40  20
```

Array becomes:

```
[20, 40, 60, 10, 30]
```

Then:

```
j = 2
```

Step 3

```
j = 2
```

```
A[j] = 60
```

Is `60 < 30` ?

No.

So only `j` moves:

```
j = 3
```

Array:

```
[20, 40, 60, 10, 30]
```

Step 4

$j = 3$

$A[j] = 10$

Is $10 < 30$?

Yes.

Increase i :

$i = 1$

Swap:

$A[1]$ $A[3]$

40 10

Array becomes:

[20, 10, 60, 40, 30]

Then:

$j = 4$

Now j has reached the pivot, so stop.

Put Pivot in Correct Position

Currently:

[20, 10, 60, 40, 30]

i pivot

We swap:

A[i+1] pivot

Here:

i + 1 = 2

So swap:

60 30

Final partition:

[20, 10, 30, 40, 60]

 pivot

Now **30 is in its correct position.**

Complete Process at a Glance

Original:

[40, 20, 60, 10, 30]

 pivot

i = -1, j = 0

40 < 30? No

 j++

20 < 30? Yes

 i++

 swap 40 and 20

[20, 40, 60, 10, 30]

60 < 30? No

```
j++
```

10 < 30? Yes

```
i++
```

swap 40 and 10

[20, 10, 60, 40, 30]

Finally:

swap A[i+1] and pivot

[20, 10, 30, 40, 60]

pivot

Then Quick Sort is applied to the two sides:

[20, 10] 30 [40, 60]

After sorting:

[10, 20] 30 [40, 60]

Final Answer

[10, 20, 30, 40, 60]

Remember These 3 Lines

```
pivot = A[high]
```

```
i = low - 1
```

```
j = low
```

Then:

```
if A[j] < pivot:
```

```
    i++
```

```
swap(A[i], A[j])
```

```
j++
```

At the end:

```
swap(A[i+1], A[high])
```

Quicksort steps

Choose pivot

Partition array

Sort left part

Sort right part

Combine

Time complexity

Case	Time
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Quicksort is often very fast in practice.

10. Mergesort

Merge Sort is a sorting algorithm based on **Divide and Conquer**.

It works in **2 main phases**:

1. **Divide** keep splitting the array into smaller parts.
2. **Merge** combine the small parts back together in sorted order.

Think:

Divide Divide Divide Merge Merge Merge

Example

We want to sort:

[40, 20, 60, 10, 30, 50]

Step 1: Divide the Array

Start with:

[40, 20, 60, 10, 30, 50]

Divide it into two halves:

[40, 20, 60] [10, 30, 50]

Now divide each half again:

[40] [20, 60] [10] [30, 50]

Divide again:

[40] [20] [60] [10] [30] [50]

Now every part contains **only one element**.

Why stop here?

A single element is already sorted.

For example:

```
[40]
```

There is nothing to sort.

Step 2: Start Merging

Now we work **backwards**.

We take two small arrays and merge them in sorted order.

Start with:

```
[20] [60]
```

Compare:

```
20 vs 60
```

`20` is smaller, so take `20` .

Then take `60` .

Result:

```
[20, 60]
```

Next:

```
[40] [20, 60]
```

We compare from the **front** of both arrays.

```
40 vs 20
```

`20` is smaller.

Put `20` into the result:

[20]

Now compare:

40 vs 60

40 is smaller.

[20, 40]

Now only 60 remains:

[20, 40, 60]

So the left half is sorted.

Step 3: Merge the Right Side

We have:

[10] [30, 50]

First merge:

[30] [50]

Compare:

30 vs 50

30 is smaller.

Then 50 .

Result:

[30, 50]

Now merge:

```
[10] [30, 50]
```

Compare:

```
10 vs 30
```

Take 10 .

Then:

```
30 vs 50
```

Take 30 .

Then take the remaining 50 .

Result:

```
[10, 30, 50]
```

Step 4: Final Merge

Now we have two sorted arrays:

```
Left: [20, 40, 60]
```

```
Right: [10, 30, 50]
```

We need to combine them.

This is the **most important part of Merge Sort**.

We compare the first unused element from each side.

Comparison 1

20 vs 10

10 is smaller.

Result:

[10]

Move right side forward.

Comparison 2

20 vs 30

20 is smaller.

[10, 20]

Move left side forward.

Comparison 3

40 vs 30

30 is smaller.

[10, 20, 30]

Move right side forward.

Comparison 4

40 vs 50

40 is smaller.

[10, 20, 30, 40]

Move left side forward.

Comparison 5

60 vs 50

50 is smaller.

[10, 20, 30, 40, 50]

Move right side forward.

Now the right side is finished.

We simply add the remaining 60 :

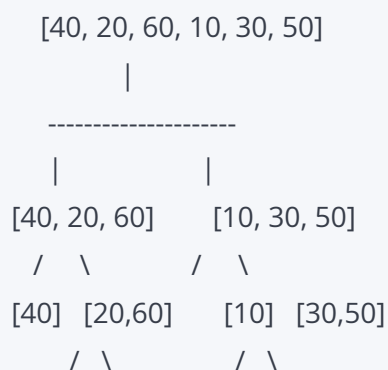
[10, 20, 30, 40, 50, 60]

Final Answer

[10, 20, 30, 40, 50, 60]

Complete Diagram

The entire Merge Sort process looks like this:



[20] [60] [30] [50]

DIVIDE

MERGE

[40] + [20,60] [20,40,60]

[10] + [30,50] [10,30,50]

[20,40,60] + [10,30,50]

[10,20,30,40,50,60]

How Does Merging Actually Work?

This is the key thing to understand.

Suppose:

Left = [20, 40, 60]

Right = [10, 30, 50]

We use two pointers:

Left: [20, 40, 60]

i

Right: [10, 30, 50]

j

Compare:

Left[i] vs Right[j]

Rule

```
If Left[i] < Right[j]
  put Left[i] into result
  move i

Otherwise
  put Right[j] into result
  move j
```

So:

```
20 vs 10  take 10
20 vs 30  take 20
40 vs 30  take 30
40 vs 50  take 40
60 vs 50  take 50
```

Then 60 is left:

```
take 60
```

Final:

```
[10, 20, 30, 40, 50, 60]
```

Merge Sort Algorithm

```
MERGESORT(A, low, high)

  if low < high

    mid = (low + high) / 2

    MERGESORT(A, low, mid)

    MERGESORT(A, mid + 1, high)
```

```
MERGE(A, low, mid, high)
```

The **MERGE** function combines two already-sorted parts.

```
MERGE(A, low, mid, high)
```

Left part = A[low ... mid]

Right part = A[mid+1 ... high]

Compare the elements from both parts.

Put the smaller element into a temporary array.

Continue until one part is finished.

Copy the remaining elements.

Copy the temporary array back into A.

Why Is It Called “Divide and Conquer”?

Because:

Divide

Break a large problem into smaller problems.

```
[40,20,60,10,30,50]
```

```
[40,20,60] [10,30,50]
```

```
[40] [20,60] [10] [30,50]
```

```
[40] [20] [60] [10] [30] [50]
```

Conquer

Sort the small pieces.

Combine

Merge them together:

[20] + [60]

[20,60]

[40] + [20,60]

[20,40,60]

And finally:

[20,40,60] + [10,30,50]

[10,20,30,40,50,60]

Time Complexity

Merge Sort has:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

This is one of its biggest advantages.

Disadvantage

Merge Sort normally needs **extra memory** to store the temporary arrays while merging.

The **most important point** is: **splitting does not sort anything; the actual sorting happens while merging.**

Time complexity

Case	Time
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

Important point

Mergesort gives **$O(n \log n)$** even in the worst case.

Its main disadvantage is that it generally requires **extra memory** for merging.

11. Heapsort

Heapsort uses a data structure called a **heap**.

For ascending order, we generally use a **max heap**.

A max heap keeps the **largest element at the top**.

A **heap** is a special type of **binary tree**.

There are two important types:

- **Max Heap** largest element is at the top.
- **Min Heap** smallest element is at the top.

Example:

```
    50
   / \
  30  40
 / \
10 20
```

Here, 50 is the largest element.

Basic idea

Suppose:

```
[40, 10, 30, 20, 50]
```

First, arrange the elements into a max heap.

The largest element becomes the root:

```
50
```

Then:

1. Remove the largest element.
2. Put it at the end of the array.
3. Restore the heap.
4. Repeat.

Eventually:

```
[10, 20, 30, 40, 50]
```

For sorting:

- **Max Heap** **Ascending order**
 - **Min Heap** **Descending order**
-

1. Max Heap

In a **Max Heap**:

Every parent is greater than or equal to its children.

Example:

```
  50
 /  \
30   40
```

```
  /\n10 20
```

Here:

```
50 > 30\n50 > 40\n30 > 10\n30 > 20
```

Therefore, it is a Max Heap.

The **largest element is always at the root.**

2. Min Heap

In a **Min Heap**:

Every parent is smaller than or equal to its children.

Example:

```
  10\n  /\n20 30\n /\n40 50
```

Here:

```
10 < 20\n10 < 30\n20 < 40\n20 < 50
```

Therefore, it is a Min Heap.

The **smallest element is always at the root.**

Heap Sort Using Max Heap

Let's sort this array in **ascending order**:

```
[40, 10, 30, 50, 20]
```

Step 1: Build a Max Heap

We rearrange the array so that the largest element comes to the top.

We get:

```
    50
   / \
  40  30
 / \
10  20
```

Array representation:

```
[50, 40, 30, 10, 20]
```

Now the largest element **50** is at the beginning.

Step 2: Move Maximum to the End

Swap the root **50** with the last element **20** :

```
[20, 40, 30, 10, 50]
```

Now **50** is in its **final position**.

We don't touch **50** anymore.

Heap portion:

```
[20, 40, 30, 10] | [50]
```

| means the element on the right is already sorted.

Step 3: Heapify Again

We need to restore the Max Heap:

[20, 40, 30, 10]

20 is smaller than 40, so swap:

[40, 20, 30, 10]

Now we have:

[40, 20, 30, 10] | [50]

Step 4: Move Maximum Again

Largest element is 40.

Swap 40 with 10:

[10, 20, 30, 40, 50]

Now:

[10, 20, 30] | [40, 50]

Heapify:

[30, 20, 10] | [40, 50]

Step 5: Move Maximum

Largest element is 30.

Swap 30 with 10 :

```
[10, 20, 30, 40, 50]
```

Now:

```
[10, 20] | [30, 40, 50]
```

Heapify:

```
[20, 10] | [30, 40, 50]
```

Step 6: Move Maximum

Swap 20 and 10 :

```
[10, 20, 30, 40, 50]
```

Final Result

```
[10, 20, 30, 40, 50]
```

So:

Max Heap gives ascending order when we repeatedly move the maximum element to the end.

Heap Sort Using Min Heap

Now let's use the same array:

```
[40, 10, 30, 50, 20]
```

We want **descending order**.

Step 1: Build a Min Heap

Make the smallest element the root:

```
    10
   / \
  20 30
 / \
50 40
```

Array:

```
[10, 20, 30, 50, 40]
```

Step 2: Move Minimum to the End

Swap 10 with the last element 40 :

```
[40, 20, 30, 50, 10]
```

Now 10 is in its final position:

```
[40, 20, 30, 50] | [10]
```

But [40,20,30,50] is not a Min Heap.

Heapify it:

```
[20, 40, 30, 50]
```

Step 3: Move Minimum

Minimum is 20 .

Swap with the last element of the heap:

[50, 40, 30, 20, 10]

Now:

[50, 40, 30] | [20, 10]

Heapify:

[30, 40, 50] | [20, 10]

Step 4: Move Minimum

Minimum is 30 .

Swap:

[50, 40, 30, 20, 10]

After continuing the process, we get:

[50, 40, 30, 20, 10]

Final Result

[50, 40, 30, 20, 10]

So:

Min Heap gives descending order when we repeatedly move the minimum element to the end.

Max Heap vs Min Heap

Feature	Max Heap	Min Heap
---------	----------	----------

Root contains	Largest	Smallest
Parent	$\geq$ children	$\leq$ children
Sorting direction	Ascending	Descending
Example root	50	10
Main operation	Extract maximum	Extract minimum

Very Important Exam Point

Remember this:

MAX HEAP

Largest Root

Move largest to END

Ascending Order

MIN HEAP

Smallest Root

Move smallest to END

Descending Order

Heap Array Formula

When a heap is stored in an array and indexing starts from 0 :

For a node at index i :

Left Child = $2i + 1$

Right Child = $2i + 2$

Parent = $(i - 1) / 2$

For example:

```
Array = [50, 40, 30, 10, 20]
```

Tree:

```
      50 (index 0)
     /   \
    40 (index 1)  30 (index 2)
   /  \
  10 (index 3) 20 (index 4)
```

For 40 , which is at index 1 :

```
Left child =  $2(1)+1 = 3$  10
Right child =  $2(1)+2 = 4$  20
```

Heap Sort Algorithm

Using Max Heap

```
HEAPSORT(A)
```

1. Build Max Heap
2. for $i = n-1$ down to 1:
 swap $A[0]$ and $A[i]$
 heapify(A, 0, i)
3. Array is sorted in ascending order

Using Min Heap

```
HEAPSORT(A)
```

1. Build Min Heap
2. for $i = n-1$ down to 1:
 swap $A[0]$ and $A[i]$
 heapify($A, 0, i$)
3. Array is sorted in descending order

Time Complexity

Both Max Heap Sort and Min Heap Sort have:

Best Case = $O(n \log n)$
Average Case = $O(n \log n)$
Worst Case = $O(n \log n)$

And the important advantage is that **Heap Sort does not require a large extra array like Merge Sort does.**

Time complexity

Case	Time
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

A major advantage is that heapsort has **$O(n \log n)$ worst-case performance** and can be performed with very little additional memory.

12. Quick Comparison of the Three Sorts

Feature	Quicksort	Mergesort	Heapsort
Main idea	Pivot & partition	Divide & merge	Heap

Best	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Extra memory	Usually low	Usually higher	Low
Main concept	Pivot	Divide and merge	Max/min heap

Easy Revision Summary

Sparse Matrix

A sparse matrix contains **mostly zeros**.

Instead of storing all zeros, we store only useful non-zero information.

Triplet	row, column, value
CSR	compressed rows
CSC	compressed columns

Limitations of Arrays

- Fixed size in traditional arrays
- Insertion can be slow
- Deletion can be slow
- Possible memory wastage
- Usually stores the same data type
- Searching can be slow if unsorted

Applications of Arrays

- **Data storage** marks, prices, temperatures
- **Image processing** pixels and RGB values
- **Financial analysis** sales, profits, expenses
- **Statistics** mean, median, standard deviation
- **Machine learning** datasets and features
- **Social media analytics** likes, comments, shares, views

Sorting

- **Quicksort** choose a pivot and partition
- **Mergesort** divide, sort, and merge
- **Heapsort** build a heap and repeatedly remove the largest/smallest element

Simple way to remember:

Quick = Pivot

Merge = Divide + Merge

Heap = Largest/Smallest at the top